Ternary Expressions

Consider the following logic:

```
$test = false;
if ($test === false) {
    $val = 'default';
} else {
    $val = $test;
}
var_dump($val);
```

```
Output:
string(7) "default"
```

It's pretty straightforward; we want to assign the value of `$test` to `$val`, but only if it has a value to be used. Otherwise, we'll give `$val` the value of `'default'`. It's a bit of a pain to write, though, and a fairly common construct, so PHP offers something called the *ternary* logical operator. So-called because it has three parts:

```
$val = ($test) ? $test : 'default';
```

The example above accomplishes the same thing as the example before it, but with far less code. The general form is `(TEST_EXPR) ? EXPR_IF_TRUE : EXPR_IF_FALSE`, any (legal) expression is acceptable in the three sections, and the entire ternary expression evaluates to `EXPR_IF_TRUE` or `EXPR_IF_FALSE` (depending on the outcome of the first `TEST_EXPR`):

```
$val = (false) ? 'whatever you like...' : 'this ' . 'could also be ' .
'anything.';
var_dump($val);
```

```
Output:
string(28) "this could also be anything."
```

Like control structures, ternary expressions can be nested, though it is generally recommended not to make a habit of it. Unreadable code will quickly follow. Where the result of `TEST_EXPR` is wanted as the result of the whole expression, you can use the short ternary expression form: